Introduction to R: Statistical Computing Program

Mvelo Mnengi

13 June 2026

1 Introduction

This report documents the implementation of a statistical computing program written in R. The program provides an interactive menu-driven interface for generating random numbers from common probability distributions, visualising data, computing descriptive statistics, calculating probabilities, approximating integrals using Monte Carlo methods, and determining quantiles.

The program is structured around a main loop that presents the user with a menu of options. Each option delegates to a dedicated function, keeping the code modular and readable. All six features are fully implemented with input validation and error handling throughout.

2 Methodology

2.1 Random Number Generation

The program generates random numbers by sampling from six probability distributions using R's built-in random generation functions. The user specifies the number of values and the required distribution parameters.

The distributions supported and their R functions are:

- Normal: `rnorm(n, mean, sd)`
- Uniform: `runif(n, min, max)`
- Exponential: `rexp(n, lambda)`
- F: `rf(n, df1, df2)`
- Chi-Squared: `rchisq(n, df)`
- T: `rt(n, df)`

2.2 Probability Calculation

Probabilities are computed using R's cumulative distribution functions. Three types of probability are supported:

$$P(X \leq x) \tag{1}$$

$$P(X \geq x) = 1 - P(X \leq x) \tag{2}$$

$$P(a < X < b) = P(X \leq b) - P(X \leq a) \tag{3}$$

All three types are available for each of the six supported distributions using `pnorm`, `punif`, `pexp`, `pf`, `pchisq`, and `pt`.

2.3 Measures of Spread

The program computes three descriptive statistics from the generated random numbers:

- Mean: $\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$
- Standard deviation: $s = \sqrt{\frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2}$
- Variance: s^2

These are computed using R's built-in `mean()`, `sd()`, and `var()` functions respectively.

2.4 Integral Approximation

Integrals are approximated using a Monte Carlo method. Given a function g, lower bound a, upper bound b, and number of samples n, the approximation is:

$$\int_a^b g(x)\,dx \approx (b-a) \cdot \frac{1}{n} \sum_{i=1}^{n} g(x_i) \tag{4}$$

where $x_i \sim \text{Uniform}(a, b)$. The user selects the function to integrate from `dnorm`, `dunif`, or `dexp`, and specifies the bounds and number of samples.

2.5 Quantile Determination

Quantiles are determined using R's built-in quantile functions. Given a probability p, the quantile x satisfies:

$$P(X \leq x) = p \tag{5}$$

The program supports quantile calculation for all six distributions using `qnorm`, `qunif`, `qexp`, `qf`, `qchisq`, and `qt`.

3 Code Structure

The system contains nine functions:

RandomMenu(): Displays the distribution selection menu for random number generation. Prints six numbered options corresponding to the supported distributions.

GetDistr_Choice(): Prompts the user to enter a distribution choice between 1 and 6 and returns it as an integer.

GenerateRandom(choice): Accepts the user's distribution choice and prompts for the required parameters. Generates and returns a vector of random numbers from the selected distribution. Returns `NULL` for invalid choices.

PlotHistogram(data): Accepts a numeric vector and plots a histogram using R's `hist()` function with a blue fill and black border.

Get_MeasuresOfSpread(random_numbers): Displays a submenu allowing the user to select mean, standard deviation, or variance. Computes and prints the selected measure from the provided numeric vector.

DistributionMenu(): Displays the distribution selection menu used within the probability and quantile functions. Identical in content to `RandomMenu()`.

GetProbabilities(): Displays a submenu for selecting the probability type ($P(X \leq x)$, $P(X \geq x)$, or $P(a < X < b)$). Prompts the user for a distribution and its parameters, then computes and prints the result using the appropriate cumulative distribution function.

ApproximateIntegrals(): Prompts the user to select a function to integrate, specify bounds, and enter the number of samples. Generates uniform random numbers over the specified interval and applies the Monte Carlo approximation formula.

DetermineQuantiles(): Prompts the user to select a distribution and enter a probability value between 0 and 1. Computes and prints the corresponding quantile using the appropriate quantile function.

MainMenu(): Prints the main menu with seven numbered options.

main(): Contains the main program loop controlled by a boolean flag `running`. Initialises `random_numbers` as `NULL` and updates it when the user generates new values. Delegates to the appropriate function based on the user's menu selection.

The program uses local variable scoping throughout. `random_numbers` is initialised and managed inside `main()` and passed explicitly to functions that require it, avoiding global assignment.

4 Expected System Behavior

4.1 Menu Options

Option 1 prompts the user to select a distribution and enter the required parameters, then generates and stores a vector of random numbers.

Option 2 plots a histogram of the generated random numbers. If no numbers have been generated yet, an error message is displayed.

Option 3 prompts the user to select a measure of spread and computes it from the generated random numbers. If no numbers have been generated yet, an error message is displayed.

Option 4 prompts the user to select a probability type and distribution, enter the required parameters, and computes the resulting probability.

Option 5 prompts the user to select a function, enter integration bounds and number of samples, then prints the Monte Carlo approximation of the integral.

Option 6 prompts the user to select a distribution and enter a probability, then prints the corresponding quantile.

Option 7 exits the program with a goodbye message.

4.2 Error Handling

If the user enters an invalid distribution or menu choice, an error message is printed and `NULL` is returned.

If the user attempts to plot or compute measures of spread before generating random numbers, the program prints a message instructing them to generate numbers first.

If the user enters a non-integer value where an integer is expected, `as.integer()` returns `NA` and the else branch handles it with an invalid choice message.

4.3 Example Output

For Option 1 (Generate random numbers) with Normal Distribution:

```
Select a distribution:
1. Normal Distribution
...
Enter your desired choice(1 - 6): 1
Enter number of values to generate: 1000
Enter mean: 0
Enter standard deviation: 1
Random numbers generated successfully.
```

For Option 4 (Calculate probabilities), $P(X \leq x)$ with Normal Distribution:

```
Select the type of probability to calculate:
1. P(X <= x)
2. P(X >= x)
3. P(a < X < b)
Enter desired choice(1 - 3): 1
You selected P(X <= x)
Enter the mean: 0
Enter the standard deviation: 1
Enter x: 1.96
Probability: 0.9750021
```

For Option 5 (Approximate integrals) with Normal Distribution:

```
Select a function to integrate:
1. Normal Distribution (dnorm)
2. Uniform Distribution (dunif)
3. Exponential Distribution (dexp)
Enter desired choice(1 - 3): 1
Enter lower bound: -1
```

```
Enter upper bound: 1
Enter number of samples: 10000
Approximate integral: 0.6832
```

For Option 6 (Determine quantiles) with Normal Distribution:

```
Enter probability (0 - 1): 0.975
Quantile: 1.959964
```

5 Limitations

5.1 Distribution Assumption

The integral approximation is limited to three density functions. Users cannot integrate arbitrary custom functions through the menu.

5.2 Single Session

Generated random numbers are not saved between sessions. Restarting the program requires regenerating values.

5.3 Fixed Distributions

The program supports six distributions. Adding additional distributions requires modifying the source code directly.

6 Technical Requirements

R 4.0+ with no external packages required.

7 Conclusion

The program successfully implements a modular, menu-driven statistical computing system in R. It covers random number generation, histogram visualisation, descriptive statistics, probability calculation, Monte Carlo integral approximation, and quantile determination across six common probability distributions. The code is structured around single-responsibility functions with consistent input handling and error messaging throughout.